南京大学本科期中考试试卷（A 卷）标准答案

课程名称：《计算机程序的构造和解释》　　姓　名：_____

学年学期：2026-2027 学年　第 1 学期　　学　号：_____

开课单位：计算机学院　　年　级：_____

考试方式：闭卷　　专　业：_____

考试时长：120 分钟；总分 100 分

题号	1	2	3	4	5	6	7	8	总分
得分									

注意事项（请仔细阅读）：

1. 试题的难度不是线性递增的，**注意时间分配**。在这项测验中，可能有些题目较难，因此你不要在一道题上思考时间太久。遇到不会答的题目可先跳过去，如果有时间再去思考。否则，你可能没有时间完成后面的题目。
2. 代码填空题的**正确答案不止一种**。代码填空题答案正确即可得分，错误也不会倒扣分。可以直接使用的 Python 内置函数（Python Built-in Functions）包括：`all`，`any`，`bool`，`dict`，`enumerate`，`filter`，`float`，`int`，`iter`，`len`，`list`，`map`，`max`，`min`，`pow`，`print`，`range`，`reversed`，`round`，`sorted`，`sum`，`zip`。
3. 你可以在“本题评价”栏给通过标记 👍 或 👎 边上的方框来给每道试题点赞或点踩。

1　WWPD: What Would Python Display? (15 pts)

本题评价		得分	
👍☐	👎☐	评分人	

For each of the expressions in the table below, write the output displayed by the interactive Python interpreter when the expression is evaluated. The output may have multiple lines. Each expression has at least one line of output.

- If an error occurs, write **`Error`**, but include all output displayed before the error.
- To display a function value, write **`Function`**. To display an iterator or generator value, write **`Iterator`**.
- If an expression would take forever to evaluate, write **`Forever`**.

Assume that you have first started `python` and executed the statements below.

```
# Function

def make_cake(n):
  def cake(x):
    if x == 0:
      return 'cake'
    else:
      print('The cake is a lie.')
      n -= x
    return cake(n)
  return cake

# Lazy Evaluation

def combine(f, xs, ys):
  for x, y in zip(xs, ys):
    yield f(x, y)
```

```
from operator import add, mul


def nats(n):
    yield n
    yield from nats(n + 1)

def facts():
    yield 1
    yield from combine(mul, nats(1), facts())

def evils():
    yield 0
    yield 1
    next_evil = evils()
    next(next_evil)
    yield from combine(add, evils(), next_evil)
```

Expression	Interactive Output
`45**2 + 1`	`2026`
`print(7, 8, 9) * 6`	`7 8 9` `Error`
`print(print('Hello'), 'SICP')`	`Hello` （1分） `None SICP` （1分）
`print(False) and 1 / 0`	`False` （1分）

`f67 = lambda x: lambda y: (print(x + y) and 67)`
`f76 = lambda x: lambda y: (76 and print (x + 2*y))`
`f67(7)(6) or f76(6)(7)`
	`13` （1分）
	`20` （1分）

`make_cake(2)` `make_cake(2)(2)`	`Function` （1分） `The cake is a lie.` （1分） `Error` （1分）

`lst1 = [1, 2, 3, 4]`
`lst2 = lst1 + [5]`
`lst1.append(5)`
 	
`lst1 == lst2`
`lst1 is lst2`
	`True` （1分）
	`False` （1分）

`evens = combine(add, nats(0), nats(0))`
`odds = combine(add, nats(0), nats(1))`
 	
`[next(evens) for _ in range(5)]`
`[next(odds) for _ in range(5)]`
 	
`mystery = combine(add, odds, evens)`
`next(mystery)`
	`[0, 2, 4, 6, 8]` （1分）
	`[1, 3, 5, 7, 9]` （1分）
	`21` （1分）

`my_fact = facts()` `[next(my_fact) for _ in range(5)]`	`[1, 1, 2, 6, 24]` （1分）
`my_evil = evils()` `[next(my_evil) for _ in range(7)]`	`[0, 1, 1, 2, 3, 5, 8]` （1分）

2 Environment Diagram (10 pts)

本题评价	得分	
👍☐ 👎☐	评分人	

Fill in the environment diagram that results from executing the code on the right until the entire program is finished, an error occurs, or all frames are filled.

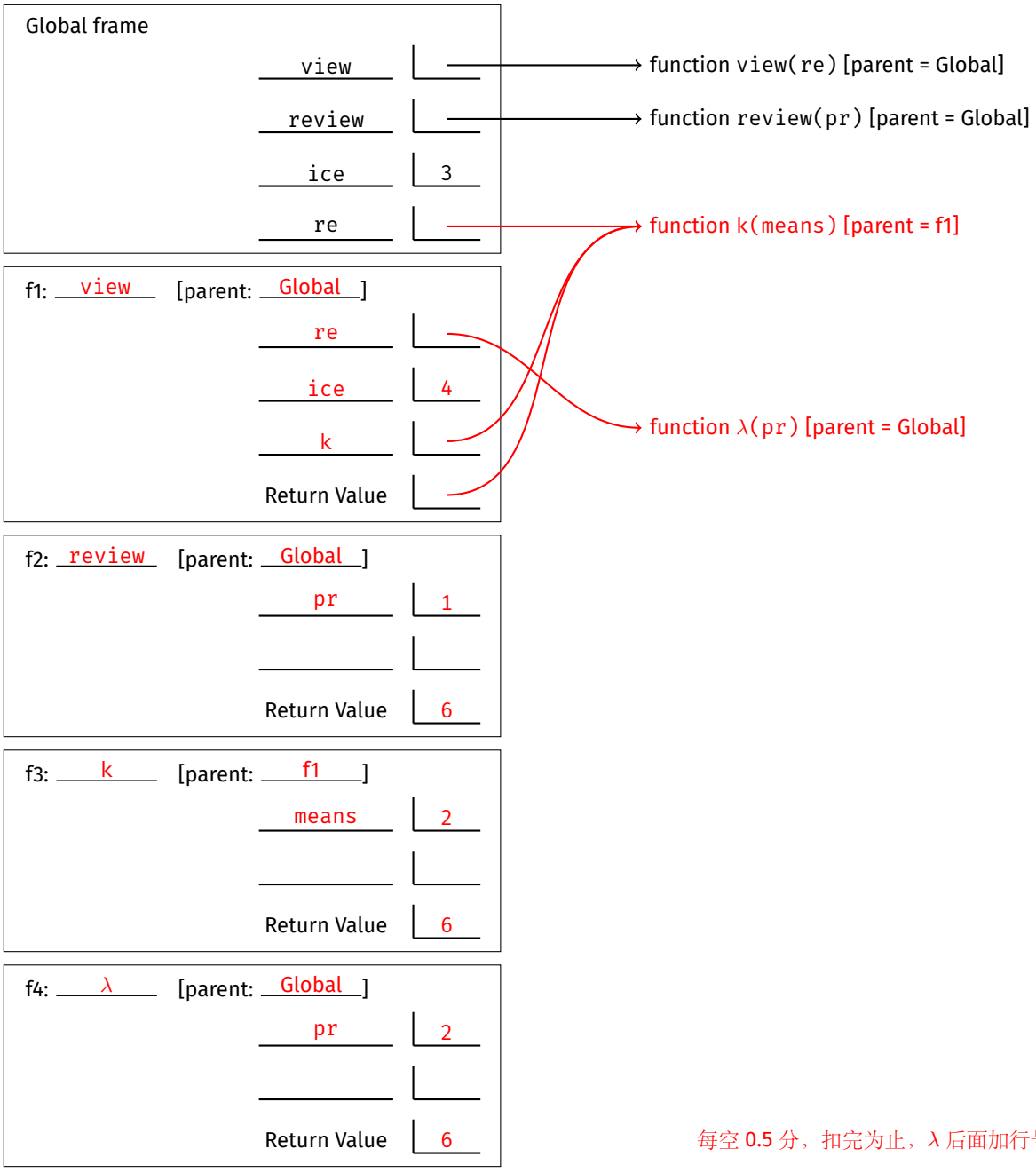
A complete answer will:

- Add missing names and parents to all local frames.
- Add missing values created or referenced.
- Show the return value for each local frame.

Hint:

- **You may not need to use all of the spaces or frames.**
- **If a variable is assigned more than once, draw its last value.**

```
1  def view(re):
2      ice = 4
3      def k(means):
4          return re(means)
5      return k
6
7  def review(pr):
8      return re(pr * 2)
9
10 ice = 3
11 re = view(lambda pr: pr * ice)
12 review(1)
```



每空 0.5 分，扣完为止，λ 后面加行号不扣分。

3 Lists (15 pts)

本题评价		得分	
👍☐	👎☐	评分人	

Kurumi falls asleep while watching the foreign exchange market. In her dream, she arrives at a potion exchange, where players buy and sell bottles of the same magic potion. All the rules are given below. Each order is a two-element list `[price, quantity]`. Both numbers are positive integers. The price is the number of gold coins per bottle, and the quantity is the number of bottles. Each order is a separate list, and the buy and sell lists are different objects. A buy order gives the highest price a buyer will pay. A sell order gives the lowest price a seller will accept.

The buy orders are already sorted by price from high to low. The sell orders are already sorted by price from low to high. Either list may be empty. For example:

```
sells = [[6, 2], [8, 3]]
buys = [[9, 3], [7, 2]]
```

To make one trade, use **the first item** from each list:

- If either list is empty, or **the highest buy price** is lower than **the lowest sell price**, stop. Make no more trades.
- Otherwise, trade the smaller of the two quantities, **at the sell price**. Subtract that quantity from both orders.
- Remove each order whose quantity is now zero.

3.1 Buys and Sells (2pts)

Start with the two lists above. After *just one* trade, what are the lists?

This trade buys 2 bottles at 6 gold coins each.

```
>>> sells
[[8, 3]] （1分）
>>> buys
[[9, 1], [7, 2]] （1分）
```

3.2 One Trade (5pts)

Complete `trade` to make *one* trade. **Assume** both lists are nonempty and the buy price is at least the sell price. Change the given lists in place and return `[price, quantity]` for this trade. The loop at the end removes orders with no quantity left.

Hint: `lst.pop(n)` removes the nth element (counting from 0).

```
def trade(buys, sells):
    """Make one trade and change both order lists."""
    quantity = min(buys[0][1], sells[0][1]) （1分）
    price = sells[0][0] （1分）
    buys[0][1] -= quantity （0.5分）
    sells[0][1] -= quantity （0.5分）
    for orders in [buys, sells] （1分）:
        if orders[0][1] == 0 （1分）:
            orders.pop(0)
    return [price, quantity]
```

3.3 Until the Market Clears (3pts)

Now make trades until no more are possible. Use `trade` from the previous part. Return a list of trades in the order they happen. This function also changes the given order lists in place. Your code must work when either input list is empty.

```
def match(buys, sells):
    """Make all possible trades.
    >>> match([[9, 3], [7, 2]], [[6, 2], [8, 3]])
    [[6, 2], [8, 1]]
    >>> match([[5, 2]], [[5, 2]])
    [[5, 2]]
    >>> match([], [[6, 2]])
    []
    """
    trades = []
    while buys and sells and buys[0][0] >= sells[0][0] （2分）:
        trades.append(trade(buys, sells) （1分） )
    return trades
```

3.4 Copying the Orders (3pts)

We want to try matching without changing the original orders. A student copies the two outer lists with slices. Fill in the outputs below. All statements run in order in the same Python session.

```
>>> buys = [[9, 3], [7, 2]]
>>> sells = [[6, 2], [8, 3]]
>>> result = match(buys[:], sells[:])
>>> buys
[[9, 0], [7, 2]] （1分）
>>> sells
[[6, 0], [8, 2]] （1分）
```

The student finds that `buys[:]` and `sells[:]` do not keep the original orders unchanged. To fix this, the student replaces them with `copy_orders(buys)` and `copy_orders(sells)`. Complete `copy_orders` below using a list comprehension. It should copy the outer list and each order, so that matching the copies will leave the original orders unchanged.

```
def copy_orders(orders):
    return [order[:] for order in orders] （1分）

>>> buys = [[9, 3], [7, 2]]
>>> sells = [[6, 2], [8, 3]]
>>> result = match(copy_orders(buys), copy_orders(sells))
>>> buys
[[9, 3], [7, 2]]
>>> sells
[[6, 2], [8, 3]]
```

3.5 The Ledger (2pts)

Finally, use list comprehensions and `sum` to find the total number of bottles traded and the total cost in gold coins. Each element of `trades` is a `[price, quantity]` list returned by `trade`.

```
def summary(trades):
    """Return [total quantity, total cost].
    >>> summary([[6, 2], [8, 1]])
    [3, 20]
    """
    total = sum([q for p, q in trades （1分） ])
    cost = sum([p * q for p, q in trades （1分） ])
    return [total, cost]
```

4 Generators (10 pts)

本题评价	得分	
👍☐ 👎☐	评分人	

In this problem, we will first build a simple splitter that reads characters and produces lines of text. The final splitter should produce lines with at most `width` characters each, including spaces.

In Python, `'\n'` represents a newline character. It **terminates** a line but does not appear in the output. For example:

- `'a\nb\n'` contains two lines, `'a'` and `'b'`. Both lines are terminated by `'\n'`.
- `'a\n\nb\n'` contains three lines: `'a'`, `''`, and `'b'`. The second `'\n'` terminates an empty line, so the second line is an empty string.
- `'a\nb'` also contains two lines, `'a'` and `'b'`. The second line is not terminated by `'\n'`, but we still keep it when the input ends.
- The empty string `''` contains no lines.

Your first task is to write a Python generator that splits `chars` into lines **without** a width limit. It should yield each line as a string:

```
def line_splitter(chars):
    r"""Yield each line as a string.
    >>> lines = line_splitter(iter("a\n\nb"))
    >>> next(lines)
    'a'
    >>> next(lines)
    ''
    >>> next(lines)
    'b'
    """
    buffer = ''
    for char in chars:
        if char == '\n':
            yield buffer （1分）
            buffer = '' （1分）
        else:
            buffer += char
    if buffer （1分） :
        yield buffer （1分）
```

The splitter above breaks lines at `'\n'`. Next, we will add a width limit so that each output line contains at most `width` characters.

To do this, first complete `width_splitter`, a Python generator that takes a string `line` and splits it into shorter lines. Assume that `width` is a positive integer and that `line` contains no `'\n'` characters.

```
def width_splitter(line, width):
    r"""Yield strings of at most width characters.
    >>> list(width_splitter("abcdefg", 3))
    ['abc', 'def', 'g']
    >>> list(width_splitter("", 3))
    ['']
    >>> list(width_splitter("abc", 3))
    ['abc']
    """
    while len(line) > width:
        yield line[:width] （1分）
        line = line[width:] （1分）
    yield line （1分）
```

Next, combine the two functions to build a splitter that handles both `'\n'` and the width limit:

```
def line_width_splitter(chars, width):
    r"""Yield each line as a string, with at most width characters.
    >>> list(line_width_splitter(iter("abc\ndefg\nhijk"), 3))
    ['abc', 'def', 'g', 'hij', 'k']
    >>> list(line_width_splitter(iter("a\n\nb\n"), 3))
    ['a', '', 'b']
    """
    for l1 in line_splitter(chars) （1分）:
        for l2 in width_splitter(l1, width) （1分）:
            yield l2
```

Finally, consider the code below. The generator `source` adds each character to `seen` before yielding it. Assume the functions above work correctly. All statements run in order in the same Python session. Fill in the output for `seen`.

```
>>> seen = ''
>>> def source():
...     global seen
...     for char in "abcdef\ngh":
...         seen += char
...         yield char
>>> reader = line_width_splitter(source(), 2)
>>> next(reader)
'ab'
>>> seen
'abcdef\n' （1分）
```

5 No Assignments? No Problem! (15 pts)

本题评价	
👍☐ 👎☐	

得分	
评分人	

Recall that in Homework 03, we're required to implement the function `pingpong`, where our evil TAs banned loops and all assignment statements. For many of us who naturally think in loops, this restriction is painful. In this problem, we will build a general **Loop-to-Recursion Translator** so that we can turn any `while` loop into pure functional recursion without ever worrying about the evil TAs' restrictions again.

5.1 The Manual Recipe and Its Trap (5 pts)

Recall the loop-to-recursion recipe from the problem's hints: to convert a `while` loop into a recursive function without assignments, we write a `helper` function that takes one parameter for each variable that changes in the loop.

First, warm up by translating `sum_squares_iter(n)` on the left into `sum_squares_rec(n)` on the right. (2 pts)

```
1 def sum_squares_iter(n):
2     total, i = 0, 1
3     while i <= n:
4         total, i = total + i * i, i + 1
5     return total
```

```
1 def sum_squares_rec(n):
2     def helper(total, i):
3         if i > n （1分）:
4             return total
5         return helper(total + i * i, i + 1) （1分）
6     return helper(0, 1)
```

However, a subtle trap appears when a loop body contains *sequential* assignments, where a later statement depends on a variable updated earlier in the same iteration. Compare `mystery_loop(n)` on the left with a classmate's recursive translation `buggy_rec(n)` on the right.

```
1 def mystery_loop(n):
2     total, i = 0, 1
3     while i <= n:
4         total = total + i
5         if total % 3 == 0:
6             i = i + 2
7         else:
8             i = i + 1
9     return total
```

```
1 def buggy_rec(n):
2     def helper(total, i):
3         if i > n:
4             return total
5         if total % 3 == 0:
6             return helper(total + i, i + 2)
7         else:
8             return helper(total + i, i + 1)
9     return helper(0, 1)
```

Evaluate the expressions on the left to see how `buggy_rec` diverges from `mystery_loop`. Then, fix `buggy_rec` on the right by replacing the condition `total % 3 == 0` on line 5 so that `buggy_rec` produces the same results as `mystery_loop`. (3 pts)

```
>>> mystery_loop(1)
1
>>> mystery_loop(2)
3 （1分）
>>> buggy_rec(2)
1 （1分）
```

```
2     def helper(total, i):
3         if i > n:
4             return total
5         if (total + i) % 3 == 0 （1分）:
6             return helper(total + i, i + 2)
7         else:
8             return helper(total + i, i + 1)
```

5.2 One Function to Run Any Loop (6 pts)

Writing a custom `helper` function for every loop is repetitive because the recursive structure is always the same. Only the loop variables, the stopping condition, the loop body, and the return expression change from problem to problem. If we pack all variables that change during the loop into a single `state` tuple, any `while` loop is determined by four components.

- `init` is the initial `state` before the loop starts.
- `cond(state)` returns `True` if the loop should continue running, or `False` if it should stop.
- `body(state)` takes the current `state` and returns the new `state` after one iteration.
- `done(state)` takes the final `state` when the loop ends and returns the final result.

A `while` loop follows the pattern below. Using the state tuple `s = (total, i)`, `mystery_loop(n)` from Section 5.1 can be rewritten with `run_loop` on the right.

```
state = init
while cond(state):
    state = body(state)
return done(state)
```

```
def mystery_with_run_loop(n):
    def cond(s):
        return s[1] <= n
    def body(s):
        if (s[0] + s[1]) % 3 == 0:
            return (s[0] + s[1], s[1] + 2)
        else:
            return (s[0] + s[1], s[1] + 1)
    def done(s):
        return s[0]
    return run_loop((0, 1), cond, body, done)
```

(3 pts) Implement `run_loop(init, cond, body, done)` to simulate the general `while`-loop pattern above using recursion, without any assignment statements or loops.

```
def run_loop(init, cond, body, done):
    """Simulate a while loop using recursion.
    >>> # Assume that mystery_with_run_loop is defined as above
    >>> mystery_with_run_loop(2)
    3
    """
    def helper(state):
        if not cond(state) (1分) :
            return done(state) (1分)
        return helper(body(state)) (1分)
    return helper(init)
```

(3 pts) Now we can finally translate the iterative `pingpong_iter(n)` below into `pingpong(n)` using `run_loop`. Assume `switch(k)` is already defined and the state tuple is `s = (k, res, d)`.

```
def pingpong_iter(n):
    k, res, d = 1, 1, 1
    while k < n:
        if switch(k):
            k, res, d = k + 1, res - d, -d
        else:
            k, res, d = k + 1, res + d, d
    return res
```

```
def pingpong(n):
    def cond(s):
        return s[0] < n (1分)
    def body(s):
        if switch(s[0]):
            return (s[0] + 1, s[1] - s[2], -s[2]) (1分)
        else:
            return (s[0] + 1, s[1] + s[2], s[2])
    def done(s):
        return s[1] (1分)
    return run_loop((1, 1, 1), cond, body, done)
```

5.3 Chaining Sequential Statements (4 pts)

In `mystery_with_run_loop(n)`, we still have to manually handle the sequential assignments. Here, we try to solve this problem by *chaining* the statements in the loop body together, so that each statement receives the state already updated by the previous one.

(2 pts) First, implement `chain(steps)`, which takes a list of one-argument functions `steps` and returns a single function that applies each function in `steps` to `state` from left to right. Do not use any assignments or loops.

```
def chain(steps):
    """Apply each function in steps to state in order.
    >>> add_one = lambda x: x + 1
    >>> double = lambda x: x * 2
    >>> f = chain([add_one, double])
    >>> f(3)
    8
    """
    def chained(state):
        if steps == []:
            return state （1分）
        return chain(steps[1:])(steps[0](state)) （1分）
    return chained
```

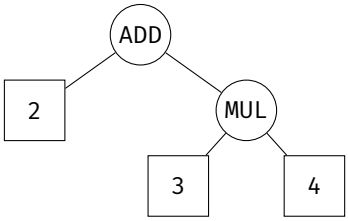
(2 pts) Now we can finally translate `mystery_loop` in 5.1 statement by statement. Use `run_loop` in 5.2 and `chain` to translate `mystery_loop`, with state `s = (total, i)`.

```
def mystery_rec(n):
    def cond(s):
        return s[1] <= n
    def step1(s):
        return (s[0] + s[1], s[1]) （1分）
    def step2(s):
        if s[0] % 3 == 0:
            return (s[0], s[1] + 2)
        else:
            return (s[0], s[1] + 1)
    def done(s):
        return s[0]
    return run_loop((0, 1), cond, chain([step1, step2]) （1分） , done)
```

6 Arithmetic Expressions (20 pts)

本题评价	得分	
👍☐ 👎☐	评分人	

An arithmetic expression can be represented by a tree. For example, $2 + 3 * 4$ can be represented by the tree on the right, where each leaf is a number and each internal node is an operator. In this problem, we will define an abstract data type (ADT) for expressions based on the tree ADT that we have learned in class. The provided operators `ADD`, `SUB`, `MUL`, and `DIV` represent addition, subtraction, multiplication, and division, respectively. The tree could be written as:




```
tree(ADD, [tree(2), tree(MUL, [tree(3), tree(4)])])
```

Recall the tree ADT: `tree(v, br=[])` constructs a tree with label `v` and a list `br` of its subtrees; `label(t)` and `branches(t)` return the label and the branches of a tree; and `is_leaf(t)` checks whether `t` is a leaf.

6.1 Implementing the Expression ADT (4 pts)

An *atom* is a leaf containing a number; a *compound* is a tree whose root is an operator and whose two branches are its left and right operand expressions. Complete the following functions that construct and access the expression ADT, with an example shown on the right.

```
def make_atom(v):
    return tree(v)

def make_comp(op, operand_l, operand_r):
    return tree(op, [operand_l, operand_r]) （1分）

def value(atom):
    return label(atom)

def operator(comp):
    return label(comp)

def operand_l(comp):
    return branches(comp)[0] （1分）

def operand_r(comp):
    return branches(comp)[1] （1分）

def is_atom(expr):
    return is_leaf(expr) （1分）

def is_comp(expr):
    return not is_atom(expr)
```

The same expression can be constructed using the expression ADT as follows:

```
make_comp(
    ADD,
    make_atom(2),
    make_comp(
        MUL,
        make_atom(3),
        make_atom(4)
    )
)
```

For an atom, `value` returns the number on the node; for a compound, `operator`, `operand_l`, and `operand_r` get access to the root operator and the two subtrees on the left and right, respectively.

6.2 Basics about the Expressions (4 pts)

Now, based on the expression ADT, implement two basic functions for the expressions, that is, `print_expr` that prints an expression ADT as a string, and `evaluate` that computes the value of an expression ADT. For this and all remaining parts, use **ONLY** the expression interface above and do **NOT** break the abstraction.

Two helpers are provided: `symbol(op)` returns the operator's symbol, and `apply(op, a, b)` applies the operator to two numbers. For example, `symbol(ADD)` returns `'+'` and `apply(MUL, 2, 3)` returns `6`.

```
def print_expr(expr):
    """Print expr the way it would be written on paper.
    >>> print_expr(make_atom(1))
    1
    >>> print_expr(make_comp(ADD, make_atom(1), make_atom(2)))
    (1 + 2)
    >>> print_expr(make_comp(ADD, make_atom(2), make_comp(SUB, make_atom(3), make_atom(4))))
    (2 + (3 - 4))
    """
    def to_string(expr):
        if is_atom(expr):
            return str(value(expr))
        left  = to_string(operand_l(expr))
        mid   = symbol(operator(expr)) （1分）
        right = to_string(operand_r(expr)) （1分）
        return '(' + left + ' ' + mid + ' ' + right + ')'
    print(to_string(expr))
```

```
def evaluate(expr):
    """Return the value of expr.
    >>> evaluate(make_atom(1))
    1
    >>> evaluate(make_comp(ADD, make_atom(1), make_atom(2)))
    3
    >>> evaluate(make_comp(ADD, make_atom(2), make_comp(SUB, make_atom(3), make_atom(4))))
    1
    """
    if is_atom(expr):
        return value(expr) （1分）
    return apply(operator(expr), evaluate(operand_l(expr)), evaluate(operand_r(expr)) （1分） )
```

6.3 Step by Step Evaluation (6 pts)

We say a compound is *ready* if both of its operands are atoms. Implement `step`, which returns a new expression where every *ready* compound in `expr` is replaced by an atom containing its value.

```
def step(expr):
    """Return a new expression that expr becomes after one step.
    >>> E = make_comp(MUL, make_comp(ADD, make_atom(5), make_atom(6)),
    ...               make_comp(ADD, make_atom(1), make_atom(1)))
    >>> print_expr(E)
    ((5 + 6) * (1 + 1))
    >>> print_expr(step(E))
    (11 * 2)
    >>> print_expr(step(step(E)))
    22
    >>> print_expr(step(step(step(E))))
    22
    >>> print_expr(E)
    ((5 + 6) * (1 + 1))
    >>> F = make_comp(ADD,
    ...               make_comp(MUL, make_comp(ADD, make_atom(3), make_atom(2)), make_atom(5)),
    ...               make_comp(SUB, make_atom(4), make_atom(1)))
    >>> print_expr(F)
    (((3 + 2) * 5) + (4 - 1))
    >>> print_expr(step(F))
    ((5 * 5) + 3)
    >>> print_expr(step(step(F)))
    (25 + 3)
    >>> print_expr(step(step(step(F))))
    28
    """
    if is_atom(expr):
        return make_atom(value(expr) （1分） )
    left, right = operand_l(expr), operand_r(expr)
    if is_atom(left) and is_atom(right) （1分） :
        return make_atom(evaluate(expr)) （2分）
    return make_comp(operator(expr), step(left), step(right)) （2分）
```

6.4 In Other Words (6 pts)

The expressions could be transformed by algebraic rules like the following:

- *Commutativity*: `(x + y)` can be rewritten as `(y + x)`; the same rule holds for multiplication.
- *Distributivity*: `(x * (y + z))` can be rewritten as `((x * y) + (x * z))`; the same rule holds with subtraction in place of addition.

Implement the two functions `swap` and `expand` applying the commutative and distributive rewrites, respectively. The rewrites happen only at the root of the expression and we do not rewrite the sub-expressions. You can assume the input expressions are always rewritable by the corresponding rule.

Remember that each function needs to return a *new expression*. The provided function `copy_expr(expr)` returns a deep copy of an expression without modifying the original.

```
def swap(expr):
    """Return a new expression in which the commutative rule is applied to expr.
    Assume expr has the shape the rule needs.
    >>> E = make_comp(ADD, make_atom(1), make_atom(2))
    >>> print_expr(E)
    (1 + 2)
    >>> print_expr(swap(E))
    (2 + 1)
    >>> evaluate(E) == evaluate(swap(E))
    True
    """
    left, right = operand_l(expr), operand_r(expr)
    return make_comp(operator(expr), copy_expr(right), copy_expr(left)) （2分）
```

```
def expand(expr):
    """Return a new expression in which the distributive rule is applied to expr.
    Assume expr has the shape the rule needs.
    >>> E = make_comp(MUL, make_atom(4), make_comp(ADD, make_atom(2), make_atom(3)))
    >>> print_expr(E)
    (4 * (2 + 3))
    >>> print_expr(expand(E))
    ((4 * 2) + (4 * 3))
    >>> evaluate(E) == evaluate(expand(E))
    True
    """
    left, right = operand_l(expr), operand_r(expr)
    inner_op = operator(right （1分） )

    x = copy_expr(left)
    y = copy_expr(operand_l(right) （1分） )
    z = copy_expr(operand_r(right))
    return make_comp(inner_op, make_comp(MUL, x, y), make_comp(MUL, x, z)) （2分）
```

7 This is True Music (15 pts)

本题评价	
👍☐ 👎☐	

得分	
评分人	

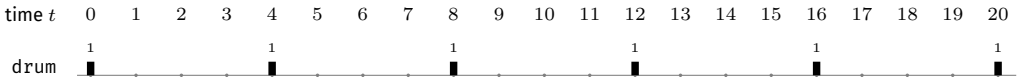
Music is a stream of sound. In this problem, we only care about the volume of the sound for simplicity. Let $\mathbb{N}$ denote the set of natural numbers and $\mathbb{R}$ denote the set of real numbers, then a stream is a function $f : \mathbb{N} \to \mathbb{R}$ where $f(t)$ is the volume of the sound at time step t.

For example, consider the stream below that regularly hits a drum to make some noise:

```
1  def drum(t):
2      if t % 4 == 0:
3          return 1
4      else:
5          return 0
```

$$\begin{array}{lll} \texttt{drum}(0) = 1 & \texttt{drum}(4) = 1 & \cdots \\ \texttt{drum}(1) = 0 & \texttt{drum}(5) = 0 & \cdots \\ \texttt{drum}(2) = 0 & \texttt{drum}(6) = 0 & \cdots \\ \texttt{drum}(3) = 0 & \texttt{drum}(7) = 0 & \cdots \end{array}$$

We can represent the stream in a sound track diagram as shown below, where the horizontal axis is the time step t and the vertical axis is the volume of the sound at that time step.

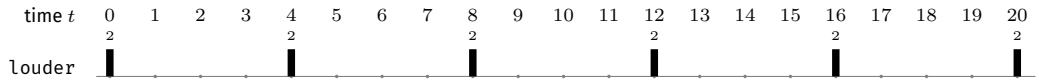


7.1 My Recording Studio (7 pts)

As the sound engineer of the band, you need to arrange, mix, and edit the sound tracks to make a great song. Let's start with some simple effects that modify a single track.

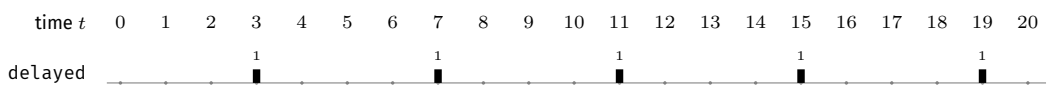
- The first effect is to adjust the overall volume of a track:

```
1  def scale(stream, factor):
2      """Return a new stream whose volume at every time t is stream(t) * factor.
3      >>> louder = scale(drum, 2)
4      >>> [louder(i) for i in range(10)]
5      [2, 0, 0, 0, 2, 0, 0, 0, 2, 0]
6      """
7      return lambda t: stream(t) * factor （1分）
```



- The second effect delays the sound track by a given number of time steps:

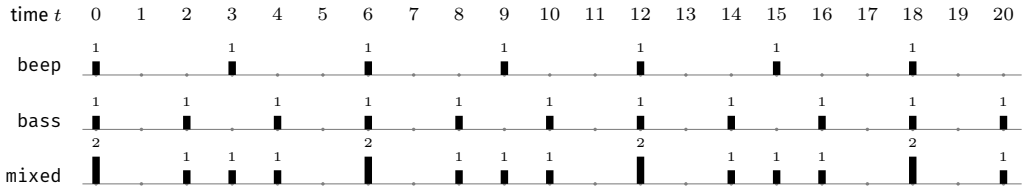
```
1  def delay(stream, offset):
2      """Return a new stream that delays stream by offset steps.
3      >>> delayed = delay(drum, 3)
4      >>> [delayed(i) for i in range(10)]
5      [0, 0, 0, 1, 0, 0, 0, 1, 0, 0]
6      """
7      def delayed(t):
8          if t < offset （1分）:
9              return 0
10         else:
11             return stream(t - offset) （1分）
12     return delayed
```



Sometimes, we need some more complex effects on the sound tracks. In such cases, we need a `lift` function that generalizes the operation over single-time-step volumes to the operation over the whole stream.

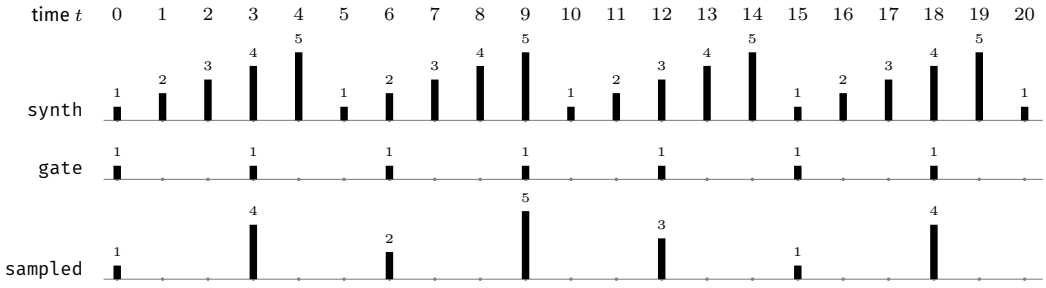
For example, in the doctests below, we lift the addition operator over integers `lambda a, b: a + b` to the mix operation `mix`, which adds up the volumes of two streams at each time step.

```
def lift(op):
    """Lift a binary operator on integers to a binary operator on streams.
    >>> beep = lambda t: 1 if t % 3 == 0 else 0
    >>> bass = lambda t: 1 if t % 2 == 0 else 0
    >>> mix = lift(lambda a, b: a + b)
    >>> mixed = mix(beep, bass)
    >>> [mixed(i) for i in range(10)]
    [2, 0, 1, 1, 1, 0, 2, 0, 1, 1]
    """
    return lambda f, g: lambda t: op(f(t), g(t)) （2分）
```



To validate your understanding of the `lift` function again, try to implement the `sample` function that samples a stream every `interval` time steps.

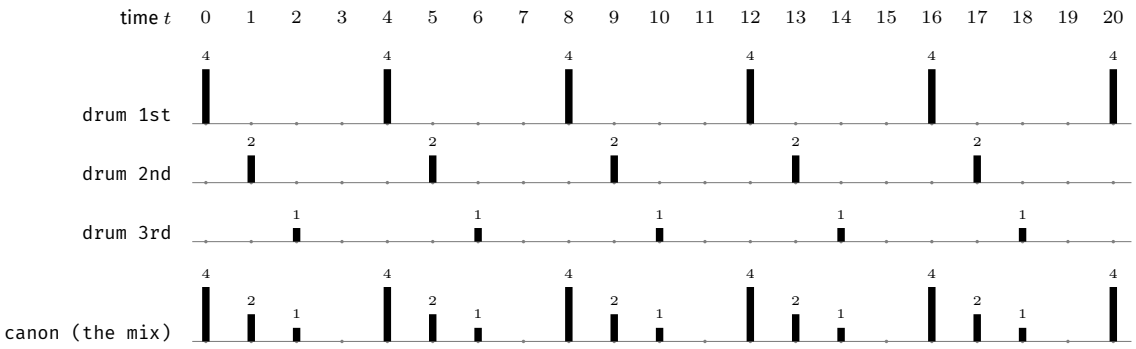
```
def sample(stream, interval):
    """Return a new stream that samples stream every interval steps.
    >>> synth = lambda t: 1 + t % 5
    >>> sampled = sample(synth, 3)
    >>> [sampled(i) for i in range(10)]
    [1, 0, 0, 4, 0, 0, 2, 0, 0, 5]
    """
    gate = lambda t: 1 if t % interval == 0 else 0
    return lift(lambda a, b: a * b （2分） )(stream, gate)
```



7.2 Canon in D, P 37 — Johann Pachelbel (5 pts)

Now we have prepared a series of tools to arrange, mix, and edit the sound tracks; let's use them to make some real noise. We can start from *canon*, also known as a round (重奏), the form of music where the same melody is played by different parts starting at different times.

For example, the following diagram shows a simple canon of the echoing drum track, where the second part is an echo (a delayed and quieter version) of the first part, and the third part is a further echo of the second part.



First, define the `echo` effect that returns a stream that is a half-volume, one-step-delayed version of the original input sound track.

```
def echo(stream):
    """Return a new stream that is a half-volume, one-step-delayed version of stream.
    >>> drum1 = lambda t: 4.0 if t % 4 == 0 else 0.0
    >>> drum2 = echo(drum1)
    >>> drum3 = echo(drum2)
    >>> [drum1(0), drum2(1), drum3(2)]
    [4.0, 2.0, 1.0]
    """
    return scale(delay(stream, 1), 0.5) （1分）
```

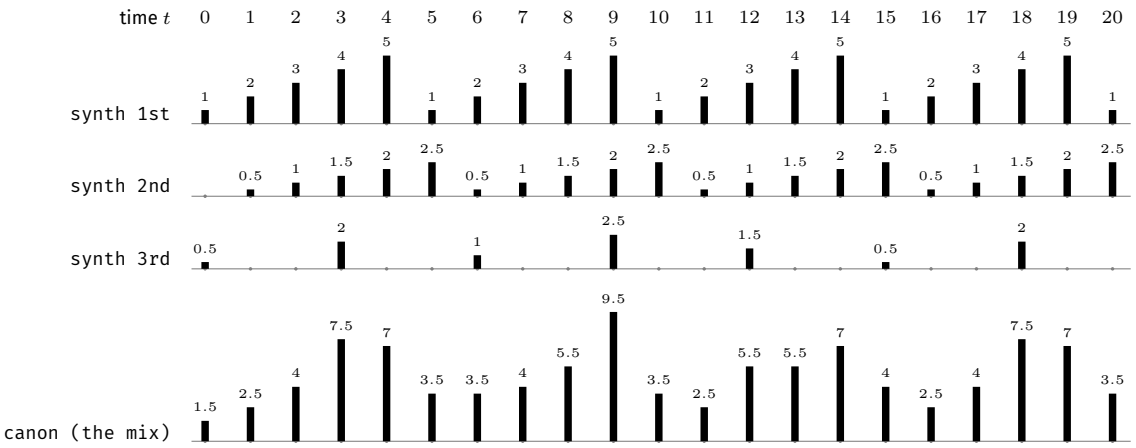
Then, consider the `echo_canon` function that mixes the first `n` parts of a canon together.

```
def echo_canon(stream, n):
    """Return the mix of n parts: stream and its first n - 1 echoes.
    >>> mix = echo_canon(lambda t: 4.0 if t % 4 == 0 else 0.0, 3)
    >>> [mix(i) for i in range(10)]
    [4.0, 2.0, 1.0, 0.0, 4.0, 2.0, 1.0, 0.0, 4.0, 2.0]
    """
    if n == 0:
        return lambda t: 0 （1分）
    return lift(lambda a, b: a + b)(
        stream,
        echo_canon(echo(stream), n - 1 （1分） )
    )
```

The echoing canon is still kind of boring here, and we can make it more interesting by setting a list of effects; we apply the effects to the same stream and mix all the applied results and the original stream together.

Implement the following `canon` function that takes a stream and a list of effects, and returns a stream that mixes the results of applying each effect in the list to the original stream plus the original stream itself.

```
def canon(stream, effects):
    """Mix stream itself with the results of applying each effect in effects to stream.
    >>> synth = lambda t: 1 + t % 5
    >>> effects = [
    ...     lambda s: scale(delay(s, 1), 0.5),
    ...     lambda s: scale(sample(s, 3), 0.5)
    ... ]
    >>> mix = canon(synth, effects)
    >>> [mix(i) for i in range(10)]
    [1.5, 2.5, 4.0, 7.5, 7.0, 3.5, 3.5, 4.0, 5.5, 9.5]
    """
    if effects == []:
        return stream （1分）
    return lift(lambda a, b: a + b)(
        effects[0](stream), canon(stream, effects[1:]) （1分）
    )
```



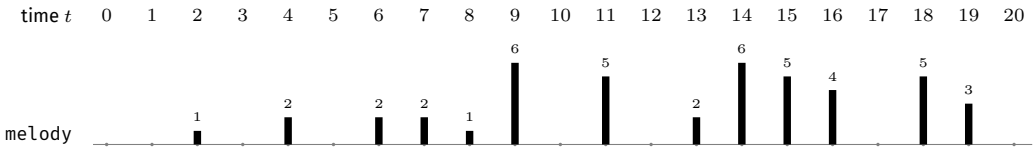
7.3 A Melody That Never Ends (3 pts)

A canon also needs a seed track worth echoing, and a drum beat or a synth loop is too simple for that.

In this problem we construct a more interesting one: Van Eck's melody. Write `melody` for the stream; it satisfies the following rules:

- If there exists some $k < t - 1$ with $\texttt{melody}(k) = \texttt{melody}(t-1)$, take the largest such k and set $\texttt{melody}(t) = t - k - 1$.
- Otherwise, $\texttt{melody}(t) = 0$.

The melody looks like the following diagram:



Implement the following `make_melody` function that returns a stream of the melody. Notice that since the current value of the melody depends on the previous values, we use a list `history` to store the previous values of the melody in the function closure. Each time we compute `melody(t)`, we first check whether it has already been computed, and if not, we traverse the history to look for the largest $k < t - 1$ with $\texttt{melody}(k) = \texttt{melody}(t-1)$, if any.

```
def make_melody():
    """Return Van Eck's melody as a stream: make_melody()(t) is melody(t).
    >>> melody = make_melody()
    >>> [melody(i) for i in range(12)]
    [0, 0, 1, 0, 2, 0, 2, 2, 1, 6, 0, 5]
    """
    history = [0]
    def melody(t):
        nonlocal history （1分）
        while len(history) <= t:
            last = history[-1]
            next_value = 0
            for k in range(len(history) - 2, -1, -1) 或 reversed(range(len(history) - 1)) （1分）:
                if history[k] == last:
                    next_value = len(history) - k - 1 （1分）
                    break
            history += [next_value]
        return history[t]
    return melody
```

8 Bonus (3 pts)

得分	
评分人	

1. Choose the correct name of our QQ group from the following options: B （1分）

   A. *SICP* - 南京大学 *2026* 秋　*B.* 南大 *SICP 2026* 秋　*C.* 南京大学 *SICP 2026* 秋季学期　*D.* *SICP* - 南大 *2026* 秋季学期

2. Name one of the awards (奖品) of this year's *Hog Contest* (you may answer in Chinese).

   奶龙、洗护套装、钢笔、螺丝刀套装、充电宝、南大小礼品 （1分）

3. What is the ID of the latest lab so far? Your answer should be in the form `labXX`, where XX is a number. lab05 （1分）